

GRUPON°2

CONTENEDORES Y ORQUESTACIÓN

DOCKER Y KUBERNETES

DOCENTE: ING. HERMES RODRIGUES RIVERO

ASIGNATURA: TECNOLOGIAS ERMEGENTES

GESTIÓN: 2026

ESTUDIANTES

1. CRISTIAN ROMAN ROSAS

2. CARLOS GABRIEL BASTO GOMEZ



CARRERA DE
INGENIERÍA
DE SISTEMAS

INDICE

Contenido

1.	INTRODUCCION	1
2.	DEFINICIÓN Y ETIMOLOGÍA	1
2.1	Docker	2
2.2	Kubernetes y Orquestación	2
3.	HISTORIA Y EVOLUCIÓN CRONOLÓGICA	2
4.	CARACTERÍSTICAS TÉCNICAS Y FUNCIONALES	3
4.1	Componentes y Ciclo de Docker	3
4.2	Contenedores Frente a Maquinas Virtuales	4
4.3	Arquitectura y objetos esenciales de Kubernetes	4
4.4	Funciones y Orquestacion	5
4.5	Ventajas, limitaciones y seguridad	5
5.	CONTENEDORES Y ORQUESTACIÓN COMO TECNOLOGÍAS EMERGENTES: CRITERIOS Y EVIDENCIAS	6
6.	RELACIÓN CON LA ASIGNATURA “TECNOLOGÍAS EMERGENTES” (INGENIERÍA DE SISTEMAS6	
6.1	Objetivos y Aprendizaje	7
6.2	Propuesta de Practica integradora	7
6.3	Conclusión	7
7.	Bibliografía	8

1. INTRODUCCION

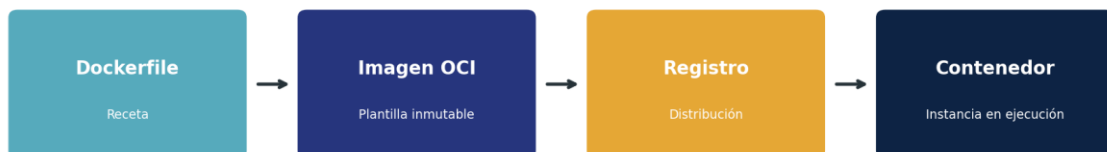
En la actualidad, el desarrollo de aplicaciones de software requiere que los programas puedan ejecutarse correctamente en diferentes computadoras y servidores. Sin embargo, una aplicación puede depender de determinadas versiones de lenguajes de programación, librerías, bases de datos y configuraciones. Cuando estas condiciones cambian entre un equipo y otro, pueden aparecer errores.

Un problema común entre desarrolladores es la frase “**en mi máquina funciona**”. Esto ocurre cuando una aplicación funciona correctamente en el equipo donde fue desarrollada, pero presenta problemas al ejecutarse en otro entorno debido a diferencias en las configuraciones o dependencias.

Para solucionar este tipo de problemas se utilizan los **contenedores**, una tecnología que permite empaquetar una aplicación junto con los elementos necesarios para ejecutarla. Una de las plataformas más utilizadas para trabajar con contenedores es **Docker**.

Por otro lado, cuando una aplicación está formada por muchos contenedores, administrarlos manualmente puede resultar complicado. En estos casos aparece la **orquestación de contenedores**, cuyo objetivo es automatizar la administración, coordinación y funcionamiento de los contenedores. Una de las herramientas más conocidas para realizar esta tarea es **Kubernetes**.

Flujo básico de contenedorización



2. DEFINICIÓN Y ETIMOLOGÍA

Un contenedor de aplicación es un proceso aislado que dispone de su propio sistema de archivos, red y recursos asignados, pero comparte el kernel del sistema operativo anfitrión con otros contenedores. Esta característica lo diferencia de una máquina virtual tradicional, que incluye un sistema operativo invitado completo. El aislamiento de un contenedor es deliberadamente más ligero; por tanto, suele iniciar más rápido y permite una mayor densidad, aunque no equivale por sí solo a una frontera absoluta de seguridad (Kubernetes, 2026a; NIST, 2017).

2.1 Docker

Docker es una plataforma y un conjunto de herramientas para desarrollar, distribuir y ejecutar aplicaciones en contenedores. El nombre se relaciona con los estibadores portuarios (dockers) y con la metáfora de transportar cargas dentro de contenedores estandarizados. Sus componentes principales incluyen Docker Engine, el cliente de línea de comandos, el daemon, imágenes, contenedores, redes, volúmenes y registros.

2.2 Kubernetes y Orquestación

Orquestar significa coordinar múltiples elementos para alcanzar un resultado. En infraestructura, la orquestación de contenedores automatiza la programación, el despliegue, la comunicación, el escalado y la recuperación de cargas distribuidas. Kubernetes proviene del griego y significa “timonel” o “piloto”; su abreviatura K8s sustituye las ocho letras centrales por el número 8. Google liberó el proyecto como código abierto en 2014 y actualmente está alojado por la Cloud Native Computing Foundation (Kubernetes, 2026a).

Docker y Kubernetes no son productos equivalentes ni rivales directos. Docker resulta especialmente útil para construir y probar imágenes y ejecutar contenedores; Kubernetes consume imágenes compatibles y administra el estado deseado de aplicaciones contenerizadas en un clúster. Desde la eliminación de dockershim, Kubernetes se comunica con runtimes compatibles mediante la Container Runtime Interface (CRI); las imágenes creadas con Docker continúan siendo utilizables por su compatibilidad con los estándares OCI.

3. HISTORIA Y EVOLUCIÓN CRONOLÓGICA

Evolución resumida de la contenedorización y la orquestación



- 1979–2000: chroot en Unix establece un antecedente del aislamiento de sistemas de archivos; posteriormente FreeBSD Jails incorpora aislamiento más amplio de procesos, red y usuarios.
- 2006–2008: Linux incorpora namespaces y control groups (cgroups), mecanismos del kernel que permiten separar procesos y limitar recursos. LXC combina estas capacidades en contenedores de sistema.
- 2013: Docker presenta una experiencia integrada para construir imágenes, transportarlas y ejecutar contenedores, simplificando la adopción por equipos de desarrollo y operaciones.
- 2014: Google publica Kubernetes, inspirado en su experiencia operando cargas distribuidas a gran escala. El proyecto se enfoca en declarar un estado deseado y reconciliar continuamente el estado real.
- 2015: se crean la Open Container Initiative (OCI), para estandarizar formatos y runtimes, y la CNCF, que recibe Kubernetes como proyecto inicial. La estandarización favorece la interoperabilidad entre herramientas.
- 2017: OCI publica las primeras versiones 1.0 de las especificaciones de imagen y runtime. NIST publica SP 800-190 con una guía de seguridad para contenedores de aplicaciones.
- 2020–2022: Kubernetes elimina progresivamente dockershim y consolida el uso de CRI con runtimes como containerd y CRI-O. Este cambio no elimina la compatibilidad con imágenes creadas mediante Docker.
- 2024–2026: Kubernetes se consolida como infraestructura de uso empresarial. En la encuesta CNCF 2025, el 82 % de los usuarios de contenedores reportó Kubernetes en producción, evidencia de madurez más que de experimentación (CNCF, 2026).

La evolución muestra dos movimientos complementarios: primero, la estandarización del paquete ejecutable —la imagen— y su aislamiento; segundo, la automatización de la operación distribuida. Esta separación permite que una misma imagen OCI sea construida, almacenada y ejecutada por distintas herramientas.

4. CARACTERÍSTICAS TÉCNICAS Y FUNCIONALES

4.1 Componentes y Ciclo de Docker

DOCKER: CONCEPTOS Y FUNCIONES CLAVE	
Función	Ejemplo o precisión
 Instrucciones declarativas para construir una imagen.	FROM, COPY, RUN, EXPOSE y CMD.
 Paquete inmutable compuesto por capas y configuración.	Puede versionarse y publicarse en un registro.
 Instancia ejecutable de una imagen.	Añade una capa escribible y un ciclo de vida.
 Almacena y distribuye imágenes.	Docker Hub o un registro privado.
 Persistencia desacoplada del ciclo de vida del contenedor.	Adecuado para datos que deben sobrevivir.
 Define aplicaciones multicontenedor en YAML.	Servicios, redes, volúmenes y configuración.

Cada instrucción relevante de un Dockerfile genera una capa reutilizable. La especificación OCI define una imagen mediante manifiesto, configuración y capas del sistema de archivos, lo que permite interoperabilidad en la construcción, transporte y preparación para ejecución (OCI, 2025a). Docker Compose, por su parte, utiliza un archivo YAML para definir los contenedores de una aplicación, sus redes y volúmenes; es apropiado para desarrollo, pruebas y escenarios multicontenedor acotados (Docker, 2026b).

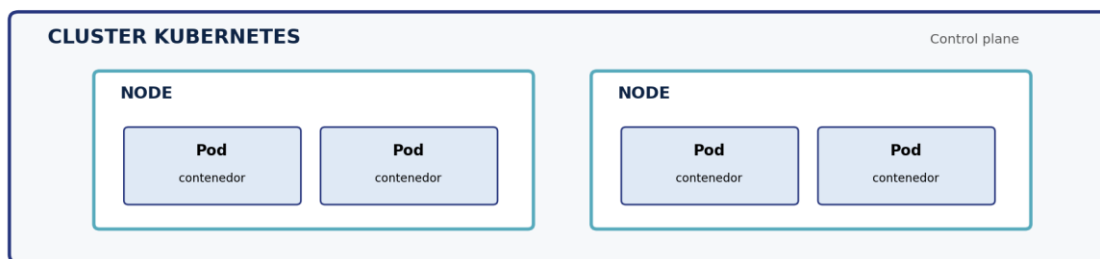
4.2 Contenedores Frente a Maquinas Virtuales

COMPARATIVA: CONTENEDOR VS MÁQUINA VIRTUAL	
Contenedor	Máquina virtual
 A nivel de sistema operativo; comparte kernel.	 A nivel de hardware; incluye SO invitado.
 Generalmente menor y rápido.	 Generalmente mayor y más lento.
 Aislamiento de procesos; depende del kernel compartido.	 Frontera más fuerte mediante hipervisor.
 Alta entre runtimes y plataformas compatibles.	 Alta, pero la imagen incluye un SO completo.
 Casos de Microservicios, CI/CD y aplicaciones empaquetadas.	 Cargas heterogéneas, SO distintos y aislamiento fuerte.

La comparación no implica que una tecnología sustituya a la otra. Es común ejecutar clústeres de Kubernetes sobre máquinas virtuales de proveedores de nube y, dentro de ellas, ejecutar contenedores. La selección depende de aislamiento, compatibilidad, costos, operación y requisitos regulatorios.

4.3 Arquitectura y objetos esenciales de Kubernetes

Arquitectura lógica simplificada de Kubernetes



- Clúster: conjunto de nodos y componentes de control administrados como una unidad.
- Control plane: mantiene el estado del clúster; incluye el servidor de API, etcd, scheduler y controladores.
- Nodo: máquina física o virtual que aporta CPU, memoria, red y almacenamiento para ejecutar cargas.

- Pod: unidad desplegable más pequeña de Kubernetes. Agrupa uno o más contenedores estrechamente relacionados que comparten red y, cuando corresponde, almacenamiento.
- Deployment: declara y actualiza un conjunto de Pods, normalmente mediante ReplicaSets, y reconcilia la cantidad de réplicas y la versión deseadas (Kubernetes, 2026c).
- Service: proporciona un punto estable de acceso y descubrimiento para uno o varios Pods, cuyas direcciones pueden cambiar a lo largo del tiempo (Kubernetes, 2026b).
- ConfigMap y Secret: separan configuración de la imagen. Los secretos requieren controles adicionales; su existencia no garantiza cifrado ni gestión segura por sí sola.
- PersistentVolume y PersistentVolumeClaim: desacoplan la necesidad de almacenamiento persistente de la vida efímera de los Pods.

4.4 Funciones y Orquestación

Kubernetes aplica un modelo declarativo: el usuario especifica el estado deseado y diversos controladores comparan continuamente ese objetivo con el estado real. Entre sus capacidades se encuentran el descubrimiento de servicios y balanceo de carga, escalado horizontal, despliegues y retrocesos controlados, asignación automática de cargas, gestión de almacenamiento, autorrecuperación y administración de configuraciones y secretos (Kubernetes, 2026a).

La autorrecuperación no significa que Kubernetes repare el código de una aplicación. Significa que puede reiniciar contenedores fallidos, reemplazar Pods, reprogramarlos en otros nodos y retirarlos temporalmente del servicio cuando no superan las verificaciones de salud. La observabilidad, el diseño tolerante a fallos y las copias de seguridad siguen siendo responsabilidades necesarias.

4.5 Ventajas, limitaciones y seguridad

Entre las principales ventajas de Docker se encuentran la consistencia entre ambientes, portabilidad, rapidez de aprovisionamiento, reutilización de capas y facilidad de integración con pipelines CI/CD. Kubernetes añade automatización, escalado, disponibilidad, actualizaciones progresivas y una API común para operar en infraestructura local, híbrida o de nube.

Sus limitaciones también deben considerarse. Los contenedores requieren una estrategia de persistencia, red, observabilidad, actualización de imágenes y control de privilegios. Kubernetes puede ser excesivo para una aplicación pequeña: introduce complejidad operativa, múltiples componentes, una curva de aprendizaje y costos de administración. La tecnología debe seleccionarse por necesidad y no por tendencia.

NIST agrupa riesgos en imágenes, registros, orquestadores, contenedores y sistema operativo anfitrión. Las prácticas recomendables incluyen usar imágenes mínimas y confiables,

analizar vulnerabilidades, firmar artefactos, evitar ejecución privilegiada, aplicar mínimo privilegio, segmentar redes, proteger credenciales, mantener actualizado el runtime y auditar configuración y actividad (Souppaya et al., 2017).

5. CONTENEDORES Y ORQUESTACIÓN COMO TECNOLOGÍAS EMERGENTES: CRITERIOS Y EVIDENCIAS

Una tecnología emergente suele caracterizarse por innovación reciente, crecimiento acelerado, capacidad de transformación y evolución técnica todavía activa. Los contenedores y Kubernetes surgieron como tecnologías emergentes durante la década de 2010 porque modificaron la forma de construir, distribuir y operar software. No obstante, en 2026 conviene formular una precisión académica: los fundamentos ya han alcanzado una adopción y madurez amplias, mientras que su ecosistema cloud native continúa evolucionando.

- Estandarización: OCI mantiene especificaciones abiertas para imágenes, runtimes y distribución. La interoperabilidad reduce la dependencia de un proveedor y permite combinar herramientas (OCI, 2025a; 2025b).
- Adopción: la encuesta anual de CNCF reportó que el 82 % de los usuarios de contenedores utilizaba Kubernetes en producción en 2025. Esto evidencia que Kubernetes pasó de tecnología experimental a infraestructura consolidada (CNCF, 2026).
- Innovación continua: GitOps, service mesh, políticas como código, eBPF, WebAssembly, contenedores confidenciales y operación de cargas de inteligencia artificial amplían el ecosistema.
- Impacto organizacional: las plataformas internas de desarrollo y las prácticas DevOps reducen fricción entre desarrollo y operaciones, pero exigen gobernanza, automatización y competencias especializadas.
- Aplicación transversal: comercio electrónico, banca, telecomunicaciones, industria, gobierno y educación emplean contenedores para servicios web, APIs, procesamiento de datos, automatización y entornos reproducibles.

Por tanto, el carácter “emergente” se encuentra hoy menos en el contenedor básico y más en la convergencia de cloud native con seguridad de cadena de suministro, edge computing, inteligencia artificial y automatización de plataformas. Esta lectura evita presentar como novedosa una tecnología que ya es estándar, sin perder de vista su evolución y su impacto.

6. RELACIÓN CON LA ASIGNATURA “TECNOLOGÍAS EMERGENTES” (INGENIERÍA DE SISTEMAS)

El tema articula contenidos centrales de Ingeniería de Sistemas: sistemas operativos, redes, desarrollo de software, arquitectura distribuida, bases de datos, seguridad y computación en la nube. Además, permite estudiar cómo una innovación técnica se convierte en un ecosistema mediante estándares, comunidades abiertas y modelos de operación.

6.1 Objetivos y Aprendizaje

- Diferenciar imagen, contenedor, runtime, registro, Pod, nodo, clúster, Deployment y Service.
- Construir una imagen reproducible mediante Dockerfile y ejecutar una aplicación multicontenedor con Compose.
- Interpretar un manifiesto YAML y desplegar, exponer, escalar y actualizar una aplicación en Kubernetes.
- Comparar contenedores, máquinas virtuales y alternativas de despliegue según criterios técnicos y económicos.
- Identificar riesgos de seguridad de la cadena de suministro y aplicar controles básicos de mínimo privilegio.

6.2 Propuesta de Practica integradora

Se propone desarrollar una aplicación web sencilla compuesta por frontend, API y base de datos. En la primera etapa se elaboran Dockerfiles, se publican imágenes versionadas y se usa Compose para ejecutar los servicios. En la segunda, la API y el frontend se despliegan en un clúster local de aprendizaje —por ejemplo, Minikube o kind— mediante Deployments y Services; luego se aumenta el número de réplicas, se simula la caída de un Pod y se ejecuta una actualización progresiva.

La evaluación puede considerar: 20 % conceptos y arquitectura; 25 % calidad y reproducibilidad de imágenes; 25 % despliegue y operación en Kubernetes; 15 % seguridad y buenas prácticas; y 15 % documentación, demostración y reflexión sobre ventajas y limitaciones. Esta práctica convierte conceptos abstractos en evidencia observable.

6.3 Conclusión

Los contenedores ofrecen un formato consistente y portable para empaquetar y ejecutar software. Docker proporciona una experiencia integrada para construir imágenes y administrar contenedores; Kubernetes mantiene aplicaciones contenerizadas en un estado declarado y automatiza funciones necesarias a escala. Trabajan dentro de un mismo ecosistema, pero resuelven problemas distintos.

La decisión de adoptar Kubernetes debe responder a necesidades reales de escalado, disponibilidad, automatización y gobernanza. Para aplicaciones pequeñas, Docker Compose o servicios administrados pueden ser suficientes. Para sistemas distribuidos, Kubernetes aporta una plataforma extensible, aunque exige inversión en conocimientos, seguridad, observabilidad y operación.

7. Bibliografia

- Cloud Native Computing Foundation. (2026). CNCF Annual Cloud Native Survey 2025: The infrastructure of AI's future. <https://www.cncf.io/reports/>
- Docker. (2026a). Docker overview. Docker Docs. <https://docs.docker.com/get-started/docker-overview/>
- Docker. (2026b). What is Docker Compose? Docker Docs. <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-docker-compose/>
- Kubernetes. (2026a). Overview. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/overview/>
- Kubernetes. (2026b). Service. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/services-networking/service/>
- Kubernetes. (2026c). Deployments. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Kubernetes. (2026d). Learn Kubernetes basics. <https://kubernetes.io/docs/tutorials/kubernetes-basics/>
- Open Container Initiative. (2025a). OCI Image Format Specification. <https://specs.opencontainers.org/image-spec/>
- Open Container Initiative. (2025b). OCI Runtime Specification. <https://specs.opencontainers.org/runtime-spec/>
- Souppaya, M., Morello, J., & Scarfone, K. (2017). Application Container Security Guide (NIST SP 800-190). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-190>
- Burns, B., Beda, J., Hightower, K., & Evenson, L. (2022). Kubernetes: Up and Running (3.^a ed.). O'Reilly Media.
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, 2014(239).